

Sistemas de Informação | AMF
Inteligência Artificial II
31 de agosto de 2026

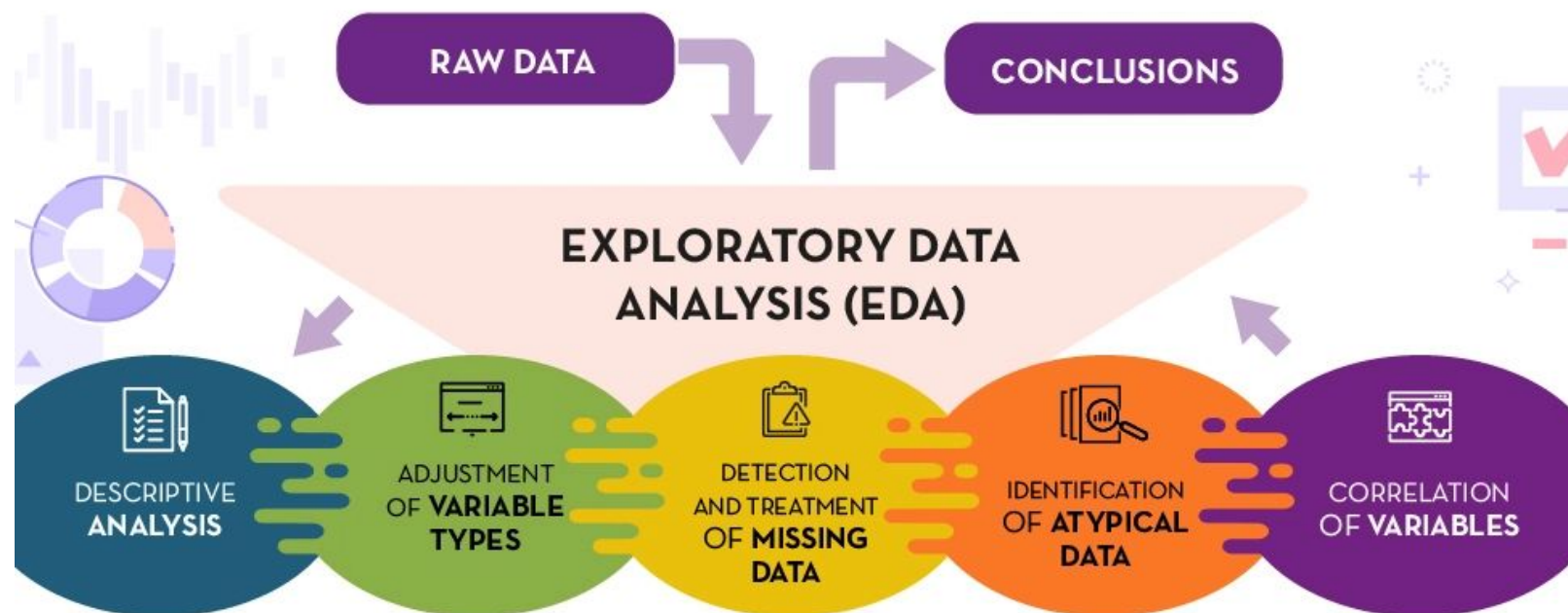
Aula 5 - Modelos tabulares e boosting

Rhauani Fazul

rhauani.fazul@amf.edu.br

Objetivo (continuação)

Exploratory Data Analysis (EDA) / Análise Exploratória de Dados



Modelos tabulares

bons atributos + modelo certo = desempenho máximo em tabulares

Modelos já vistos:

KNN → baseado em distância, funciona bem em dados pequenos.

Decision tree → regras interpretáveis, mas pouco estáveis.

SVM → ótimo em espaços de alta dimensão, depende de kernel.

Limitação: sozinhos, podem não capturar toda a complexidade de dados tabulares grandes.

Modelos tabulares

Agora, vamos aprofundar em outros algoritmos especializados com técnicas de *ensemble learning*¹:

Bagging/Bootstrap aggregation (Bagging Classifier / Regressor, Random Forest, Extra Trees, Rotation Forest, etc.)

Boosting (AdaBoost, Gradient Boosting, XGBoost, etc.)

¹ (aprendizado em conjunto) é uma abordagem de ML onde vários modelos são combinados para formar um modelo mais robusto e preciso do que cada um individualmente.

Random Forest

Intuição:

“Vários juízes independentes votando ao mesmo tempo”.

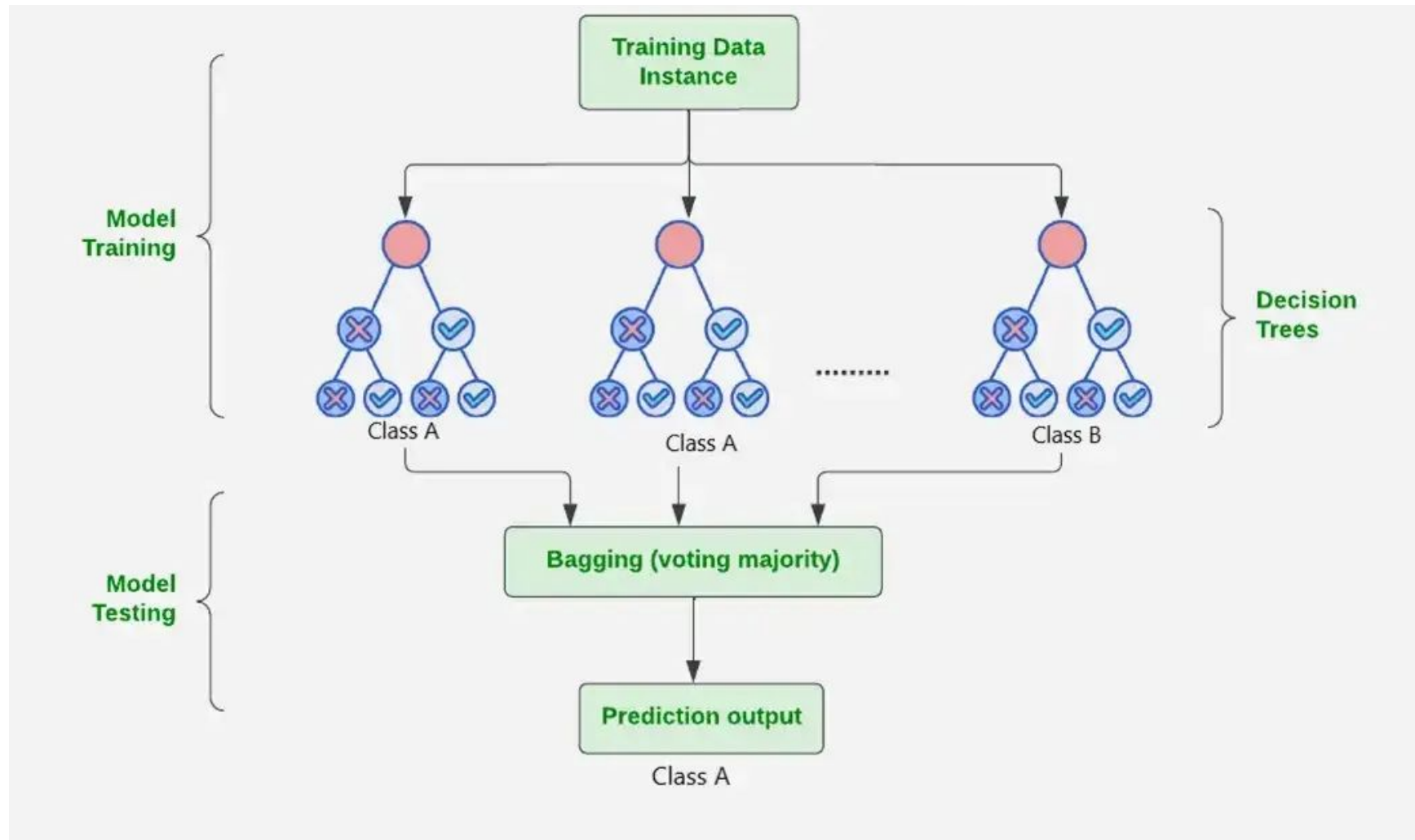
Conjunto de árvores, treinadas em **amostras diferentes**.

Cada árvore vota → agregação por maioria/média.

Objetivo: estabilidade (diminuir overfitting).

Bagging: treina vários modelos em **paralelo**, de forma independente, e depois agrega → foca em reduzir variância.

Random Forest



Random Forest

Resolve **instabilidade** das árvores individuais.

Vantagem: simples, robusto, pouco tuning.

Desvantagem: pode perder precisão máxima em problemas complexos, já que as árvores são independentes.

Boosting

Intuição:

“Um comitê de especialistas, cada novo membro aprende com os erros dos anteriores”.

Conjunto de árvores treinadas em **sequência**.

Cada nova árvore corrige os erros da anterior.

Foco em reduzir viés (aprender padrões difíceis).

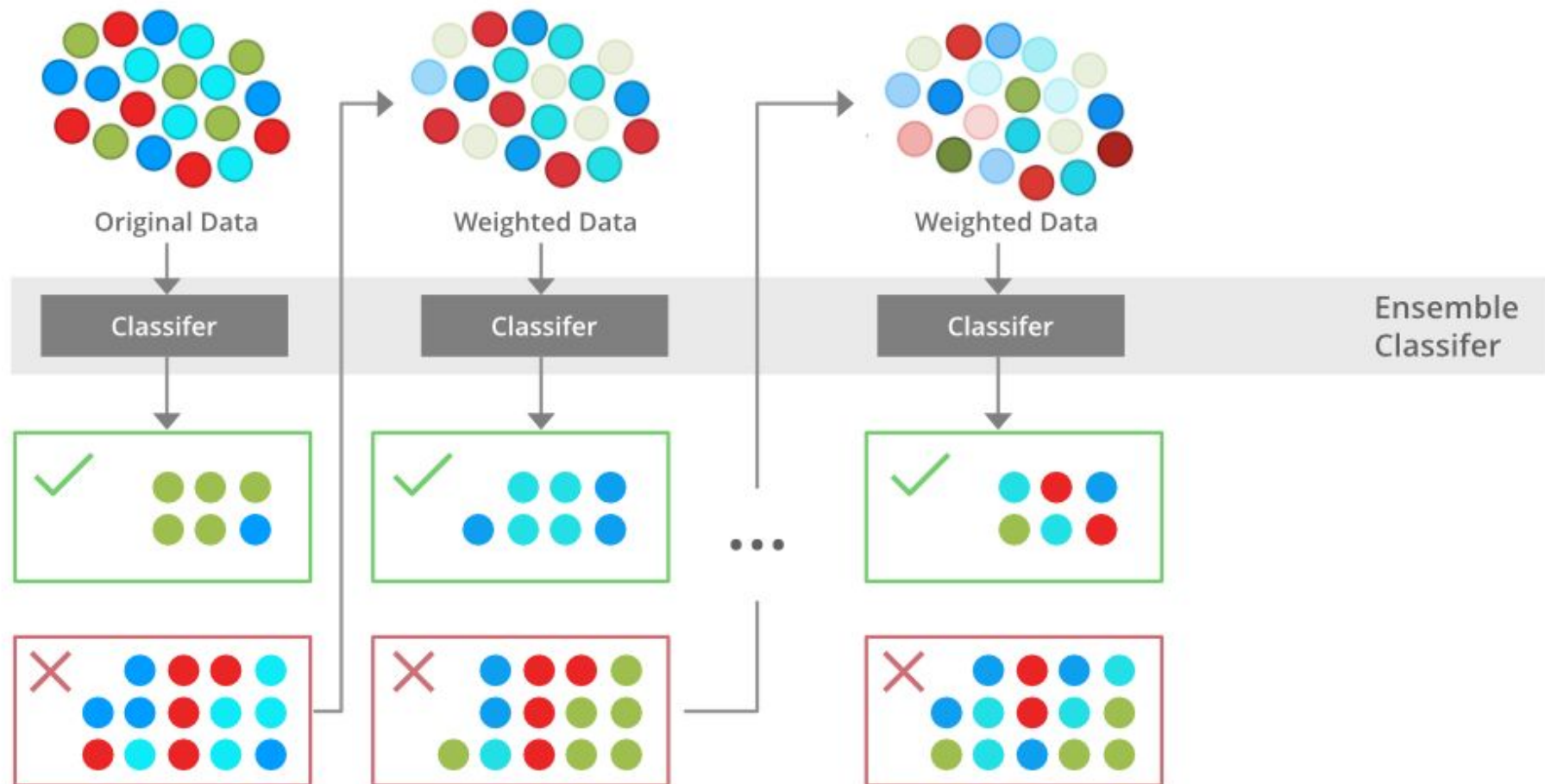
Boosting

AdaBoost → pesos maiores para exemplos mal classificados.

Gradient Boosting (GBM) → aprende (ajusta gradualmente) para reduzir erro residual.

XGBoost, LightGBM, CatBoost → versões otimizadas, com regularização e maior eficiência.

Boosting



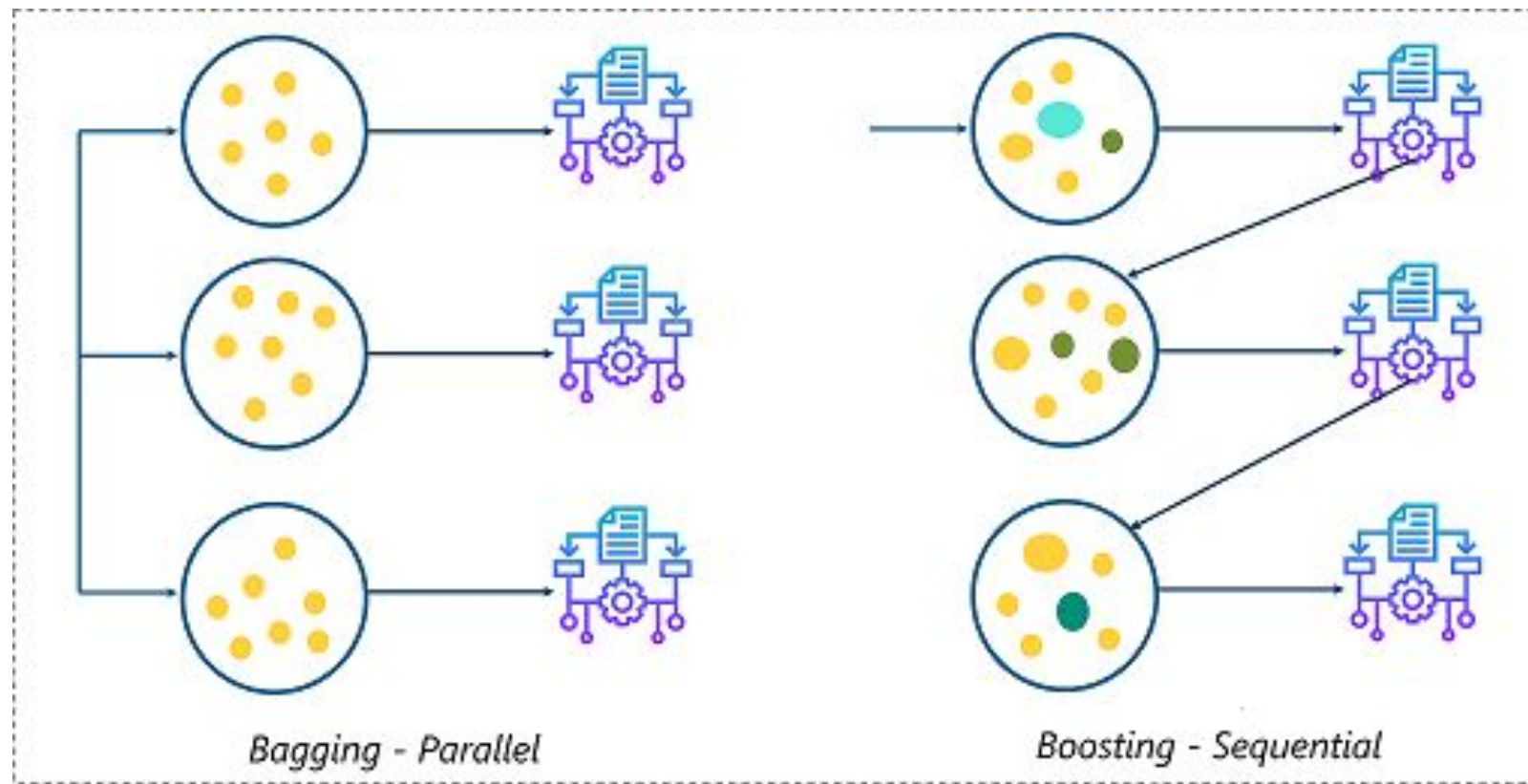
Boosting

Random Forest: mais simples, robusto a ruído, menos sensível a hiperparâmetros.

Boosting: melhor performance em tabulares, flexível, pode lidar com dados desbalanceados.

Geralmente alcançam **acurácia e performance maior** que Random Forest em dados tabulares, mas podem ser mais sensíveis a *overfitting* e tuning.

Bagging e Boosting



Bagging = vários modelos independentes + voto/média → foca em reduzir variância.

Boosting = sequência de modelos, cada um aprendendo com os erros do anterior → foca em reduzir viés.

Prática!



Explorar o dataset Titanic.



Realizar pré-processamento de dados (limpeza, encoding, normalização, feature engineering).



Escolher 1 algoritmo de ML para prever a sobrevivência.

Regressão Logística (interpretação simples, baseline).

KNN, Árvore de Decisão (visual e intuitiva), SVM.

Random Forest (robusto, mostra importância de variáveis).

Gradient Boosting (XGBoost/LightGBM/CatBoost) (mais poderoso).

Prática!

Testar quais variáveis mais impactam a sobrevivência (correlação, *feature importance*).

Correlação = foto de uma relação linear simples (uma variável $\leftrightarrow$ alvo).

Feature importance = quanto a variável ajuda na previsão quando combinada com as demais.

Fazer separação treino/teste!

(Por que separar o dataset?)

Objetivo: estimar generalização sem viés e sem vazamento.

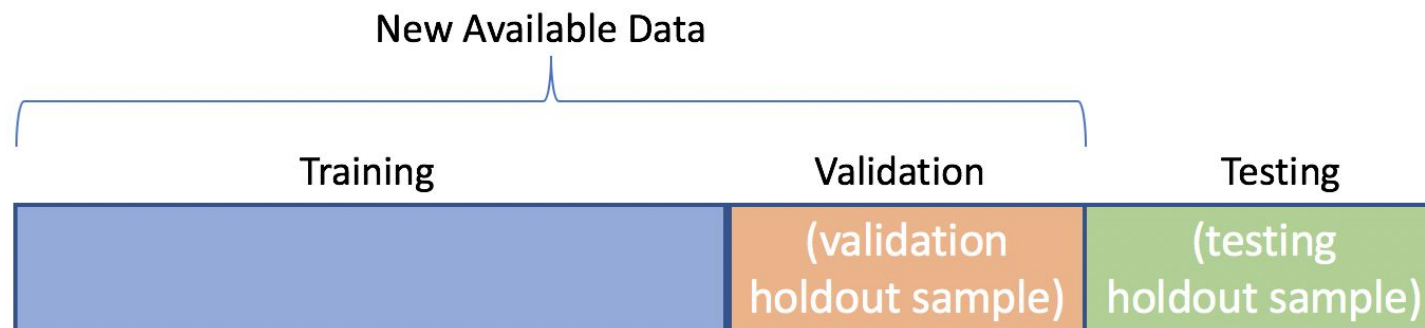
Treino: ajusta parâmetros do modelo

Validação (ou cross-validation no treino): escolhe hiperparâmetros

Teste (hold-out): avaliação fina

Teste é “sagrado”: nunca usar para tuning!

(Opção A: Train/Validation/Test)



Data which we use to **design** our models (**Training** set)

Data which we use to **refine** our models (**Validation** set)

Data which we use to **test** our models (**Testing** set)

Divide em três partes (ex.: 70/15/15):

Treino → usado para ajustar os pesos do modelo.

Validação → usado para escolher hiperparâmetros, comparar modelos.

Teste → avaliação final, como “simulação do mundo real”.

Usado quando:

Você não vai usar cross-validation.

O dataset é grande o suficiente para “gastar” uma parte só para validação.

(Opção B: Train/Test com CV)



*Data which we use to **design** our models (**Training** set)*

*Data which we use to **test** our models (**Testing** set)*

Divide o dataset em **treino** e **teste** (ex.: 80/20).

Usado quando:

- Dataset não é muito grande.
- Você vai usar **cross-validation** para simular o papel da validação.

O conjunto de **teste** só é usado **no final**, para avaliar a performance do modelo treinado/ajustado.

(Opção B: Train/Test com CV)

Se você usa **cross-validation** (k-fold) → não precisa de um conjunto de validação fixo.

O conjunto de treino é particionado em várias divisões (*folds*). Cada *fold* já desempenha o papel de “validação”.

Nesse caso, normalmente usamos só train + test.

Fluxo típico:

Separa train e test (ex.: 80/20).

Faz cross-validation dentro do *train* para escolher hiperparâmetros.

Avalia o modelo escolhido no *test*.

Treino interno (cross-validation): escolher o melhor modelo.

Teste externo (hold-out): avaliar a generalização.

(Cross-validation)

Ao invés de depender de uma única divisão, usar cross-validation dá uma avaliação mais confiável e pode mostrar ganhos pequenos que não aparecem em um único split:



Prática!

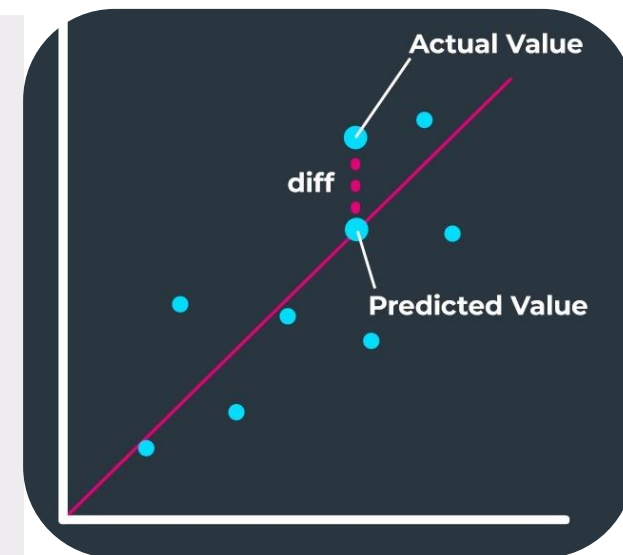
Calcular acurácia (e outras métricas que julgar relevante)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$



$$RSME = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{N - P}}$$

Reflexão

Ver: `1_rforest.py`

Correlação

Mede **relação linear entre duas variáveis**.

Exemplo: `sex_male` tem correlação de -0.54 → ser homem diminui bastante a chance de sobreviver (linearmente forte).

Mas variáveis como `fare` (quanto a pessoa pagou) têm correlação menor (0.25).

Só captura relações lineares simples.

Não leva em conta interações entre variáveis.

Reflexão

Ver: `1_rforest.py`

Feature importance

Mede quanto cada variável contribui para **reduzir o erro de previsão nos *splits*** das árvores.

RandomForest considera:

Relações não lineares (não é só correlação).

Interações entre variáveis (ex.: “idade baixa + tarifa alta → mais chance de sobreviver”).

Distribuição e dispersão das variáveis.

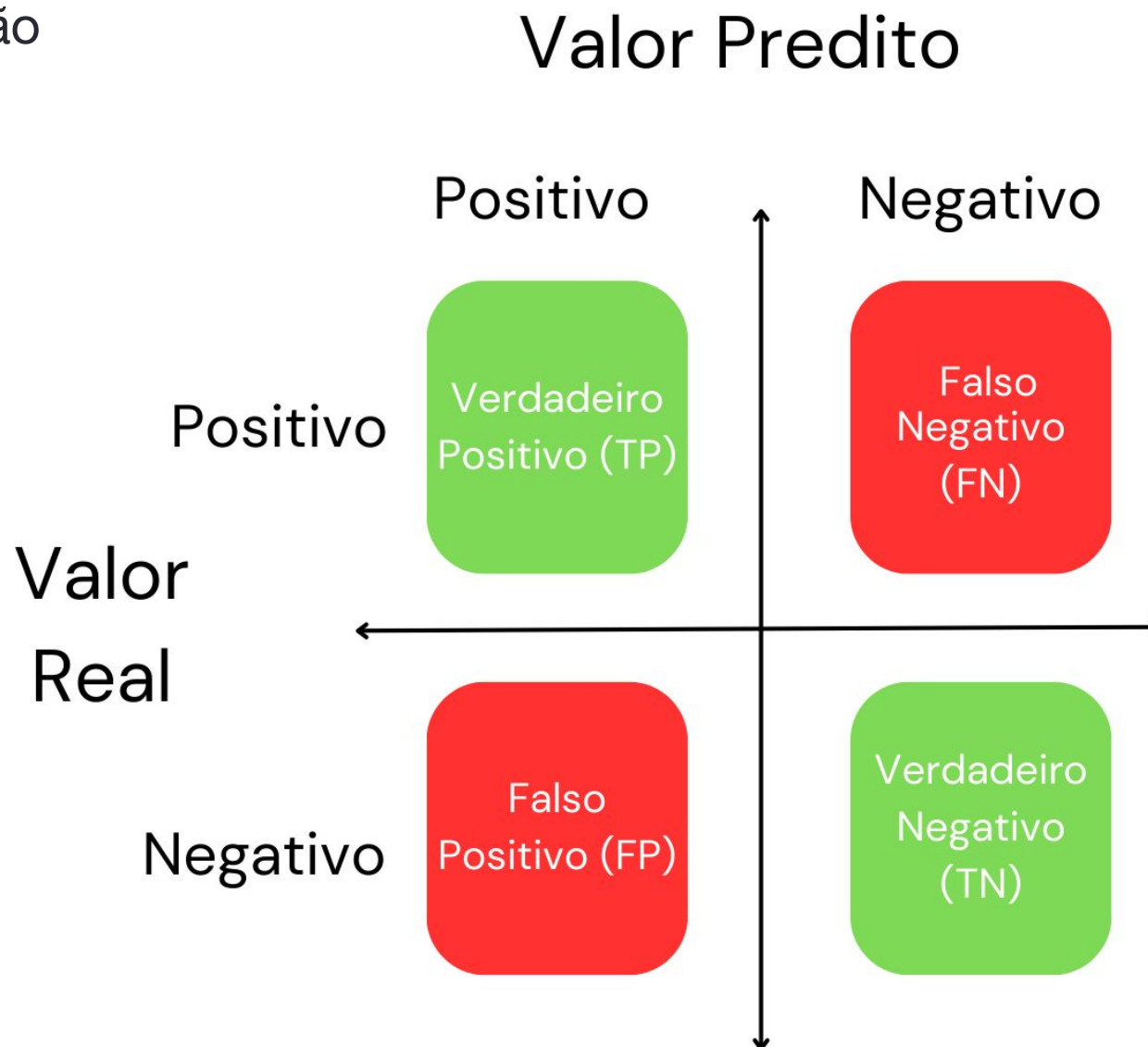
Reflexão

Como melhorar a acurácia?

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

Reflexão

Matriz de confusão



Reflexão

- **Ajustar hiperparâmetros**
- **Balanceamento de classes**
 - O Titanic tem classes desbalanceadas (sobreviventes $\approx 38\%$). Isso pode enviesar o modelo.
- **Aprimorar engenharia de atributos**
- **Testar outro ensemble**
 - GradientBoostingClassifier por exemplo
- **Cross-validation** (em vez de train/test fixo)

Ver: `1_gboosting.py`

Reflexão

Cuidado com ***data leakage*** (vazamento):

Risco de o **modelo de ML acessar informações do conjunto de teste** (ou de validação) antes da hora, o que faria a avaliação ficar artificialmente boa.

Exemplos:

Imputar valores faltantes usando a média global de todo o dataset (inclusive do teste).

Normalizar (z-score) usando média e desvio padrão do dataset completo em vez de calcular só no treino.

Criar atributos que usam o rótulo (y) (exemplo: taxa de sobrevivência por passageiro → se calculado no dataset todo, entrega a resposta).

Isso é como o aluno “dar uma espiada na prova” antes da correção.

Reflexão

Engenharia de atributos *row-wise* (linha a linha)

Quando você cria atributos apenas a partir da própria linha/observação (sem olhar o resto do dataset), não há risco de vazamento.

Exemplos no Titanic:

$\text{is_child} = \text{age} < 12$ (usa só a idade do passageiro da linha).

$\text{family_size} = \text{sibsp} + \text{parch}$ (usa só os campos já daquela pessoa).

$\text{fare_per_person} = \text{fare} / (\text{family_size} + 1)$ (usa só atributos daquela linha).

Nenhum desses precisa de estatísticas globais → logo, não vaza.

Dataset-wise (usando todos os dados): risco de vazamento → precisa ser feito dentro do pipeline/cross-validation, só com os dados de treino.

Reflexão

Cuidado com pré-processamentos (antes do split) do tipo:

```
df["age"].fillna(df["age"].median(), inplace=True)
```

A mediana calculada incluiu valores do teste, que o modelo não deveria conhecer.

→ Isso é considerado um **vazamento sutil** (mild data leakage).

Geral:

Se o valor usado para preencher/transformar um dado depende de toda a base, isso pode ser leakage.

Se o valor é calculado apenas no treino e depois aplicado ao teste, não há problema.

Reflexão

No Titanic (ou datasets relativamente simples), esse tipo de “leakage leve” normalmente não muda a acurácia de forma significativa, porque:

- A mediana/mean da idade no treino e no dataset todo não vai ser muito diferente.

- Não é uma feature diretamente ligada à sobrevivência.

Mas em cenários mais sensíveis (financeiro, médico, etc.), pode dar diferença grande.

Exemplo: preencher renda faltante usando média de toda a base (inclusive dos que você quer prever) pode alterar padrões e inflar métricas.

Reflexão

Aplicando mediana apenas no *split* de treino:

```
# split antes de tratar faltantes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# calcular mediana/moda só no treino
age_median    = X_train["age"].median()
fare_median   = X_train["fare"].median()
embark_mode   = X_train["embarked"].mode()[0]

for data in [X_train, X_test]:
    data["age"].fillna(age_median, inplace=True)
    data["fare"].fillna(fare_median, inplace=True)
    data["embarked"].fillna(embark_mode, inplace=True)
```

Reflexão

Aplicando mediana apenas no *split* de treino:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
imputer = SimpleImputer(strategy="median")
```

```
imputer.fit(X_train) # calcula só no treino
```

```
X_train = imputer.transform(X_train)
```

```
X_test = imputer.transform(X_test)
```

Usando o Pipeline do scikit-learn com SimpleImputer garante-se que o cálculo é feito só no treino, e o pipeline replica no teste.

Reflexão

Extra: testar Optuna ou BayesianOptimization

GridSearchCV

Cria todos os modelos possíveis com as combinações do param_grid

Faz validação cruzada automática em cada um deles

Calcula a média das métricas

Escolhe o melhor modelo

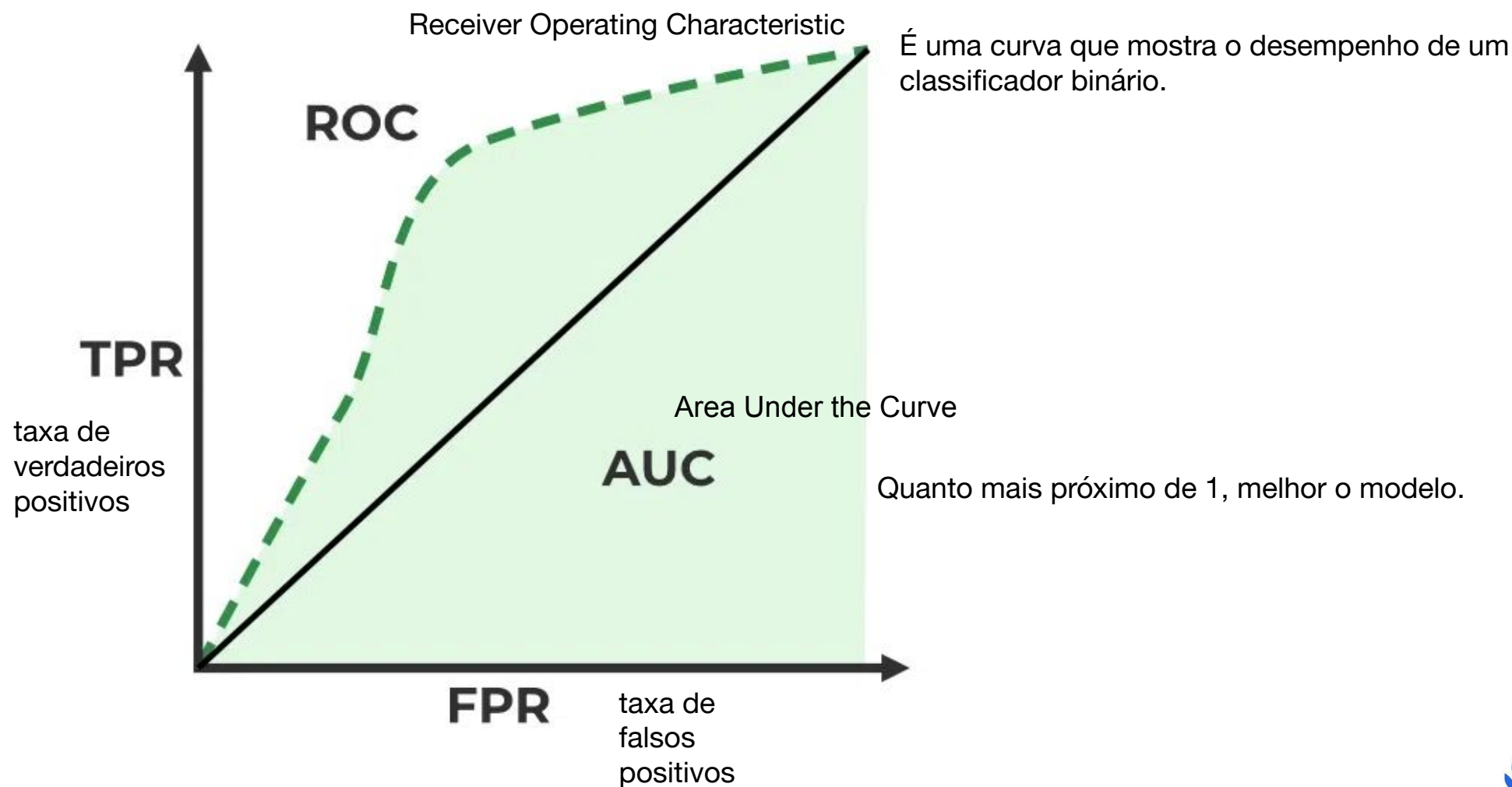
E ainda ajusta esse melhor modelo no final com todo o treino

VotingClassifier

3 modelos (XGB, RF, LR)

Ver: `1_xgboost.py`

Reflexão



Reflexão

Acurácia/erro em treino vs teste

- **Overfitting:** desempenho muito alto no treino (ex. 95%+) e bem menor no teste (ex. 70%).
- **Underfitting:** desempenho baixo em ambos (ex. treino 65%, teste 60%).
- O simples *gap* entre treino e teste já é um bom indicador.

Reflexão

Curvas de aprendizado (*learning curves*)

- Gráfico da métrica (ex. acurácia ou f1) conforme aumenta o número de exemplos de treino.
- Se as curvas de treino e validação convergem em valores baixos → **underfitting**.
- Se há um grande gap entre treino (alto) e validação (baixo) → **overfitting**.

Reflexão

Curvas de validação (*validation curves*)

- Métrica em função da complexidade do modelo (profundidade da árvore, n° de vizinhos no KNN, C no SVM etc.).
- Permite visualizar onde o modelo começa a sobreajustar ou subajustar.

Reflexão

Cross-validation scores

- Rodar `cross_val_score` e avaliar a média e o desvio padrão.
- Se a performance varia muito entre folds, pode indicar overfitting (modelo instável).

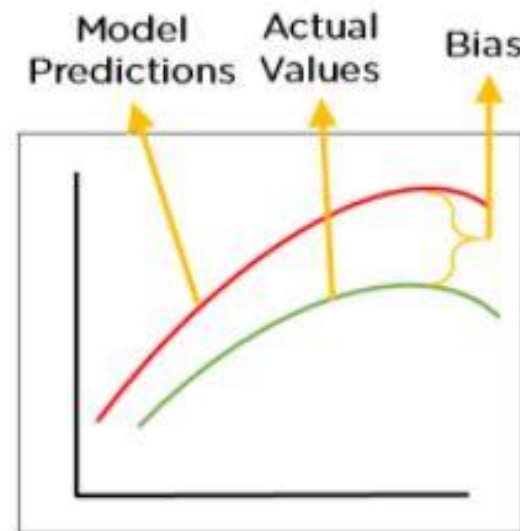
Reflexão

Métricas derivadas

Bias vs Variance tradeoff:

- **Underfitting:** alto bias (erros sistemáticos).
- **Overfitting:** alta variância (modelo se adapta demais a ruídos).
- Pode ser analisado comparando erros médios em treino e validação.

Bias é a diferença entre a predição média do modelo e o valor verdadeiro.



Próxima aula...

- Interpretação de modelos com SHAP e LIME.
- Clustering.
- Aula prática com implementação de algoritmos clássicos (K-means, DBSCAN).